Slides: Restricted access to CS 422 Fall 2026; not to be shared online or in public forums

Slides adapted from Prof. Stefan Schmid's course at TU Berlin

Kernel & Kernel Bypass - eBPF & RDMA

CS 422: Computer Networks

Instructor: Vamsi Addanki

TAs: Youngsuk Kim, Xiao Luo, Albert Vo

Fall 2026

PURDUE UNIVERSITY | Department of Computer Science

What is eBPF?

- Technology to program and modify a running (Linux) kernel
- Programs are compiled to BPF bytecode and …
  - … executed in a privileged sandbox in the kernel
- Originated from a network packet filtering system
  - BPF stands for Berkley Packet Filter
  - e for “extended”
  - But today, eBPF is a standalone term
- Flexible alternative to Linux Kernel Modules or recompiling the kernel

What is eBPF used for?

- Observability/Tracing
  - Inspect function arguments
  - Measure performance
- Networking
  - Inspect and change packet contents
  - Packet filtering, routing
  - Congestion control algorithms
- Security
  - Restrict access to resources
  - Monitoring

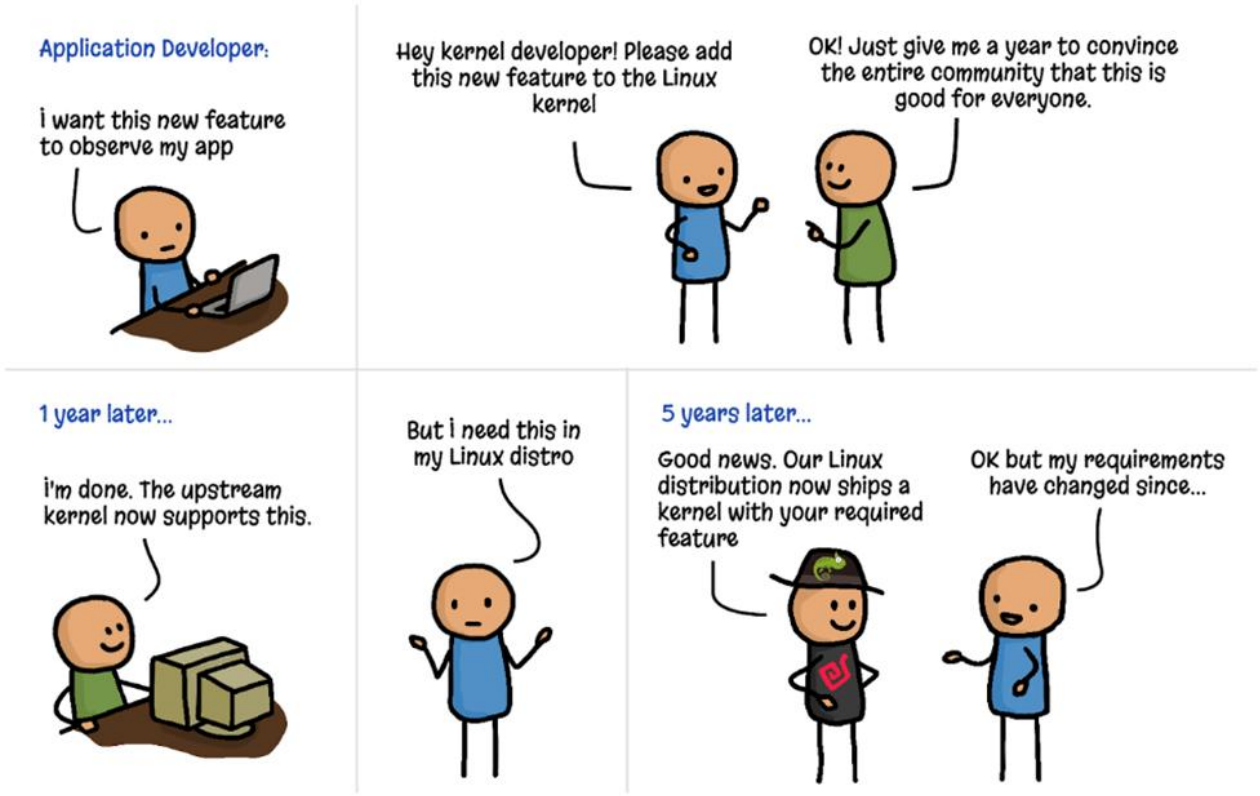
eBPF in the Real World

- Facebook's Load Balancer [1]
- Facebook's Encryption [2]
  - Cut off non-encrypted traffic
- Cloudflare's Firewall [3]
- Netflix's Observability [4]
  - Detect and analyze performance issues
- Google's Traffic Shaping [5]

eBPF in Research Community

- Accelerating memcachd [6]
- Accelerating iptables [7]
- Transparently accelerating network stack [8]
- PowerTCP in eBPF [9]

Kernel Programming: Kernel Development



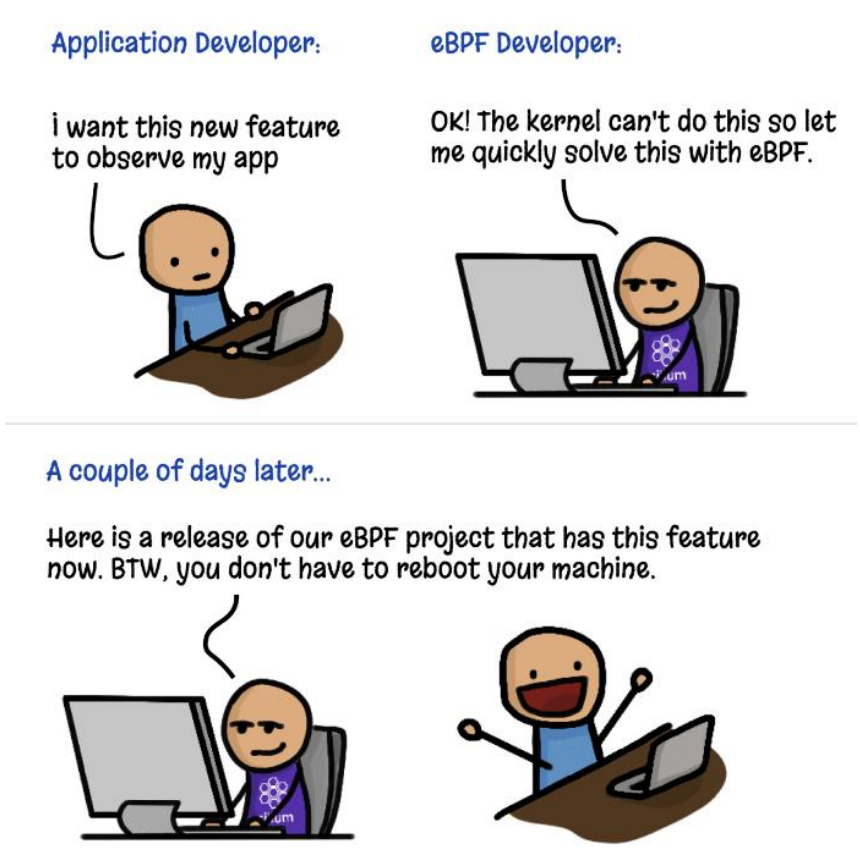
[Vadim Shchekoldin, Isovalent]

- Change kernel source code
- Convince community
- Wait several years
- Error-prone

Kernel Programming: Kernel Modules

- Kernel modules can be loaded at runtime
- Out-of-tree modules may break after new kernel releases
  - Higher maintenance
- Still error-prone
- May compromise security

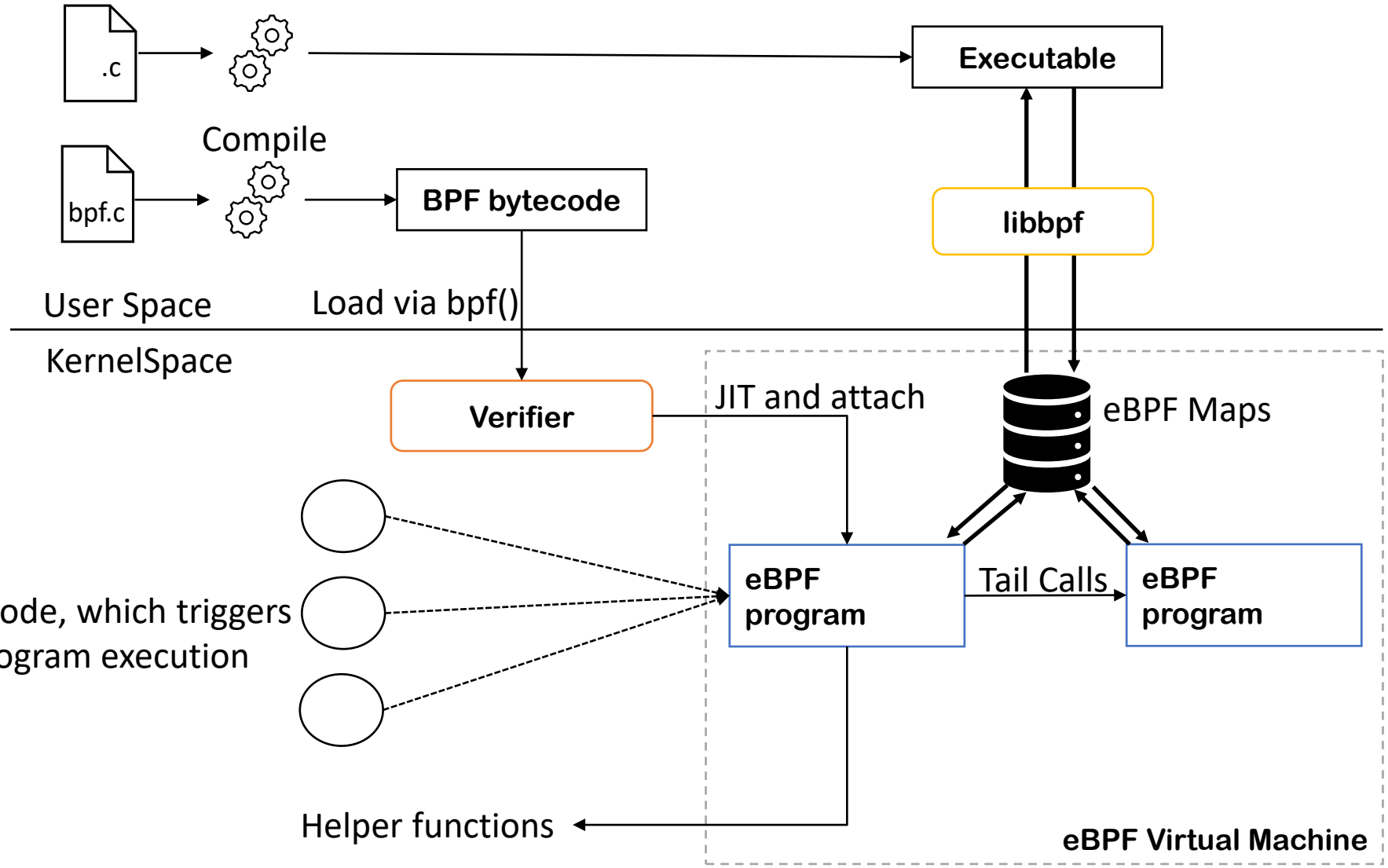
Kernel Programming: eBPF



[Vadim Shchekoldin, Isovalent]

- eBPF programs are loaded at runtime
- Safe and secure
- High performance

eBPF Architecture



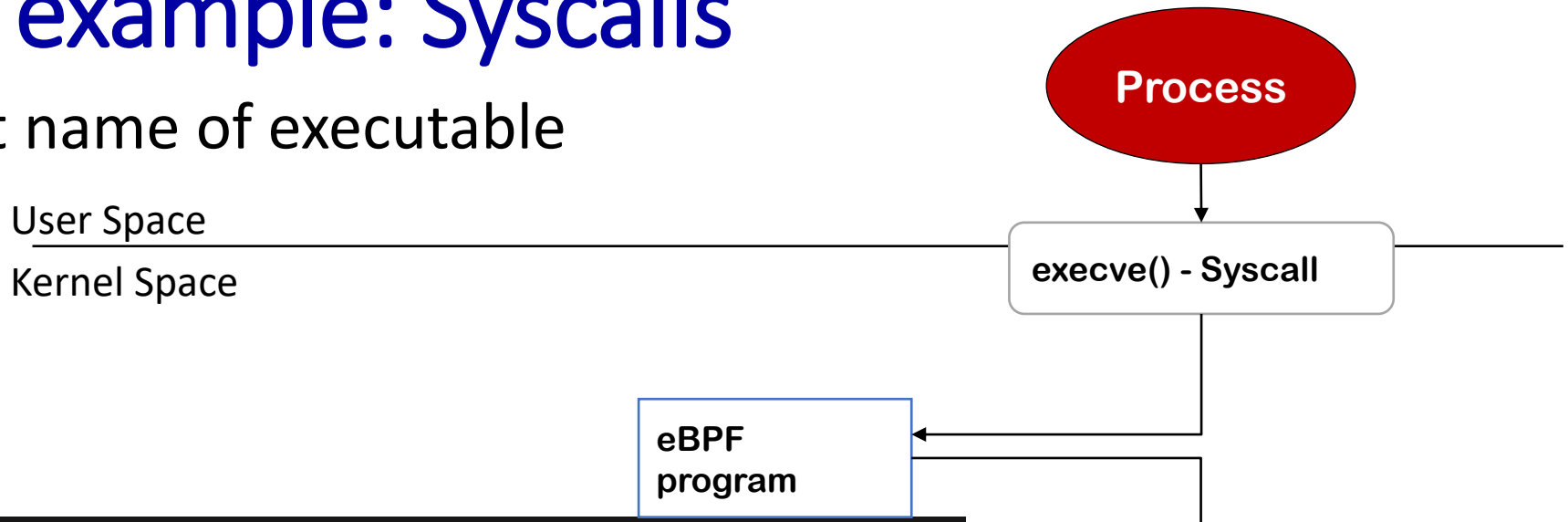
Hooks:
Kernel code, which triggers eBPF program execution

How are eBPF programs executed?

- eBPF programs are attached to certain code points (hooks)
- Example of hooks:
  - Syscalls
  - Function calls
  - Packet processing path
- As the kernel executes a hook, the linked eBPF program is called

First example: Syscalls

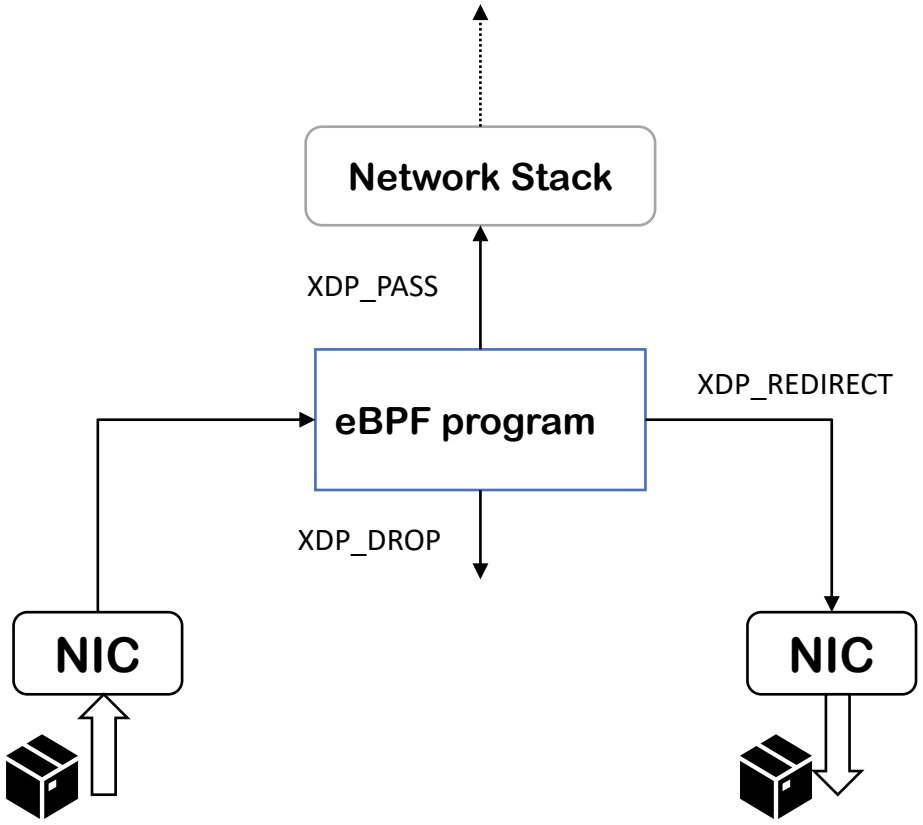
- Print name of executable




```
SEC("ksyscall/execve")
int BPF_KSYSCALL(my_probe, const char *filename)
{
    bpf_printk("Execve: %s\n", filename);
    return 0;
}
char LICENSE[] SEC("license") = "GPL";
```

Second example: Networking

- XDP: express data path
- Drop all IPv4 packets




```c
SEC("xdp")
int drop_ipv4(struct xdp_md *md) {

    void *data = (void *)(long) md->data;
    void *data_end = (void *)(long) md->data_end;
    struct ethhdr *eth = data;

    if (data + sizeof(struct ethhdr) > data_end) {
        return XDP_DROP;
    }

    if (bpf_ntohs(eth->h_proto) == ETH_P_IP) {
        return XDP_DROP;
    }

    return XDP_PASS;
}
char LICENSE[] SEC("license") = "GPL";
```

Loading C programs into the kernel?

What can possibly go wrong?

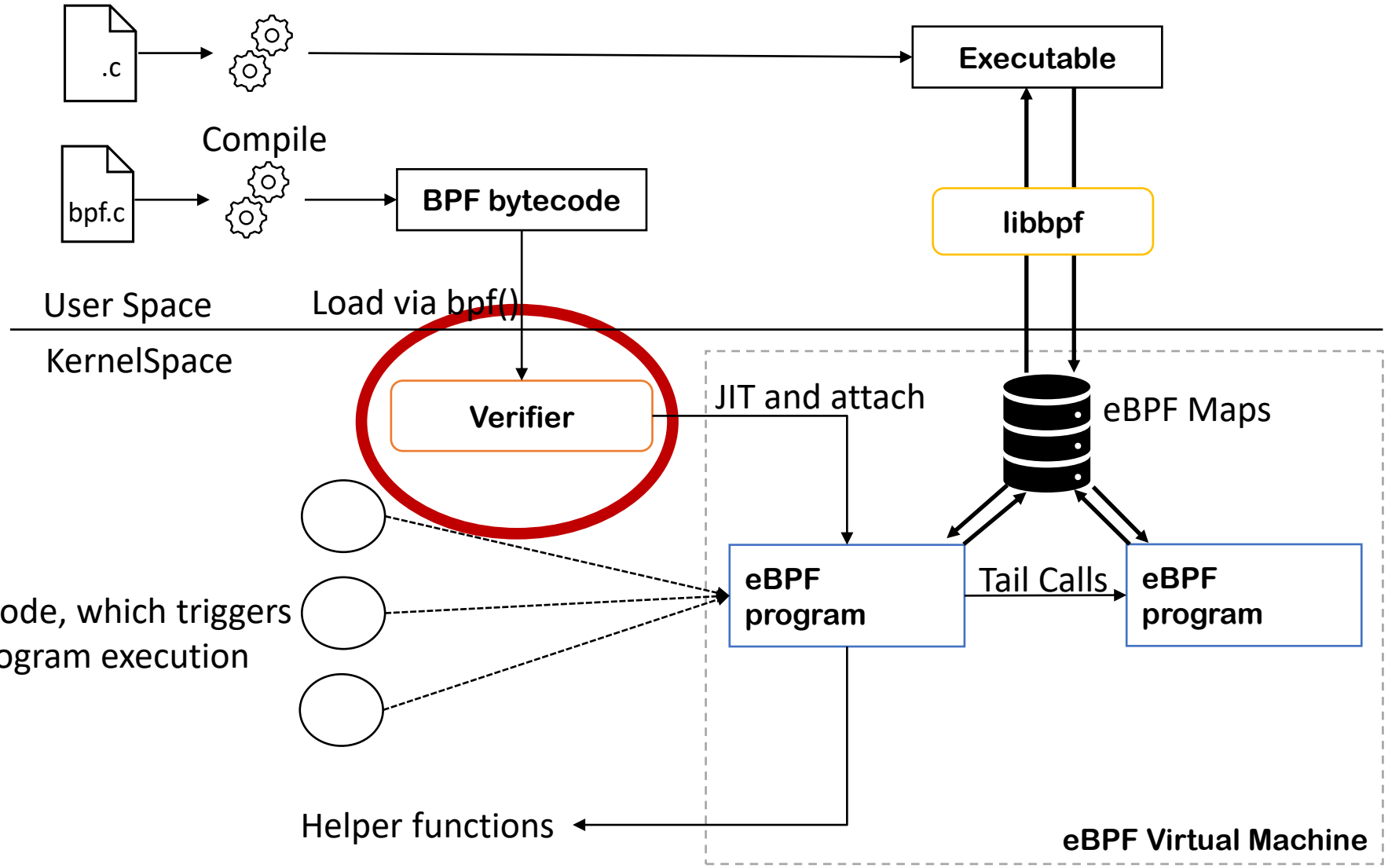
Loading C programs into the kernel?

```
SEC("ksyscall/execve")
int BPF_KSYSCALL(my_probe, const char *filename)
{
    const int *x = NULL;
    bpf_printk("%i\n", *x);
    return 0;
}
```

How to prevent crashing the kernel?

How to ensure program termination?

eBPF Verifier

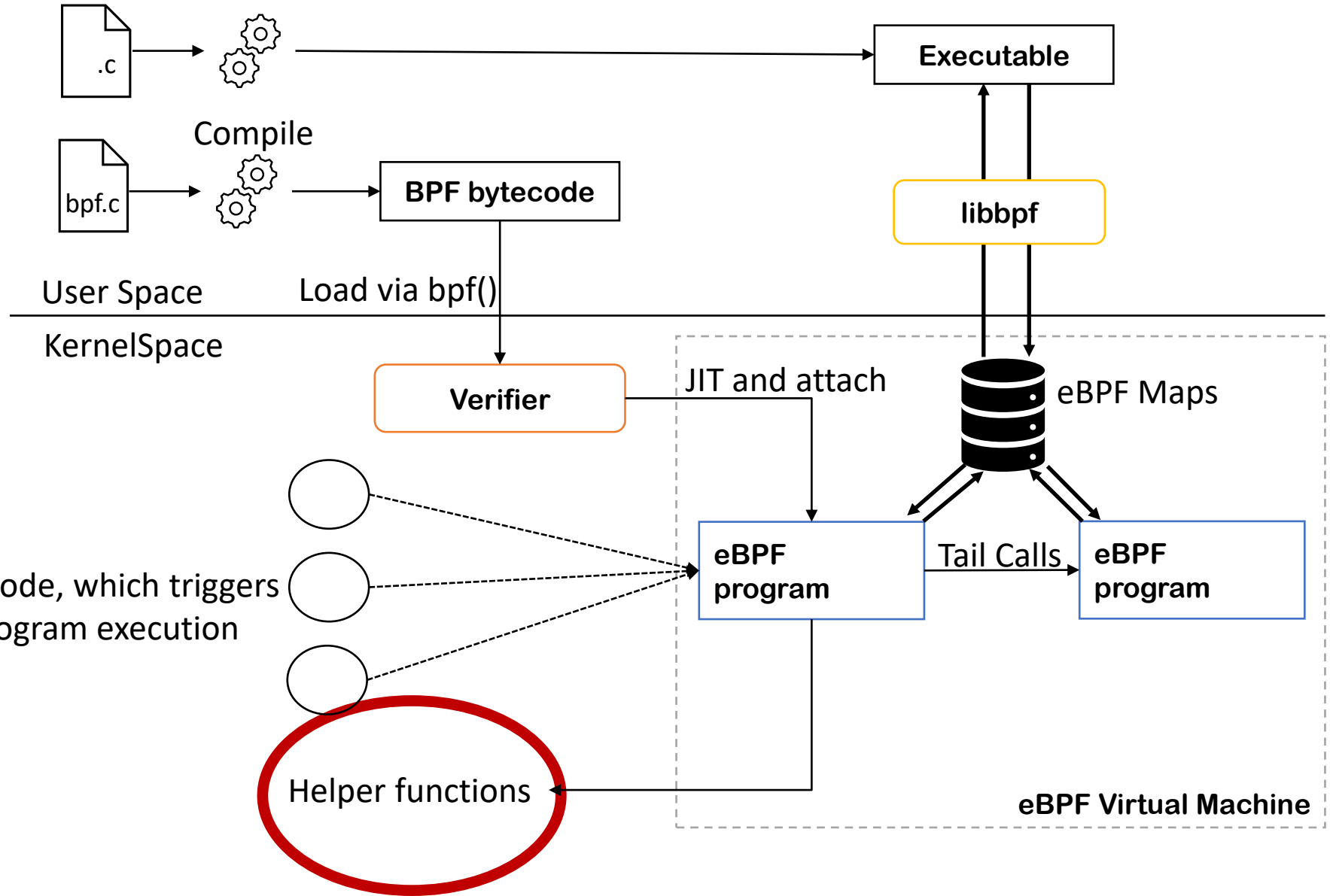


Hooks:
Kernel code, which triggers eBPF program execution

eBPF Verifier

- Analyzes all possible execution paths
- Checks memory accesses
- Checks pointer dereferencing
- Checks if programs halts
  - **Bounded loops**
- Checks return code
- Checks invalid and unreachable instructions
- Checks license
  - If using GPL-licensed bpf-helper functions …
  - … the eBPF program must also be GPL

eBPF Helper Functions

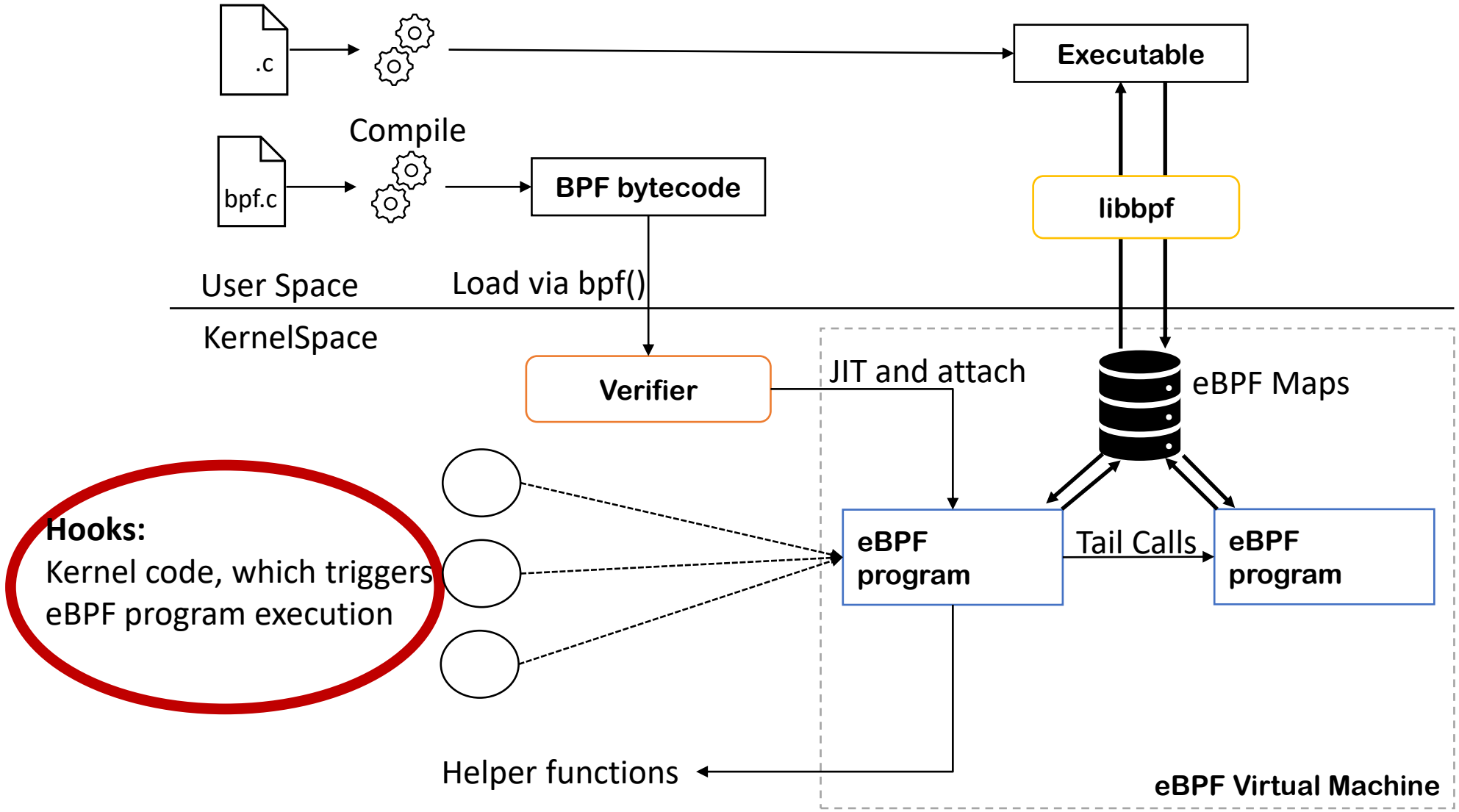


Hooks:
Kernel code, which triggers eBPF program execution

Bpf-helper functions

- Allow eBPF programs to interact with the kernel
  - Print information:
    `long bpf_trace_printk(const char *fmt, u32 fmt_size, …)`
  - Get the time in ns since system boot:
    `u64 bpf_ktime_get_ns(void)`
  - Get the total size of a given xdp buff:
    `u64 bpf_xdp_get_buff_len(struct xdp_buff *xdp_md)`
  - Get the UID associated with an event:
    `u64 bpf_get_current_uid_gid(void)`
- Check bpf-helper(7) or eBPF Docs
- Not all helper-functions are available for every eBPF program type

eBPF Program Types



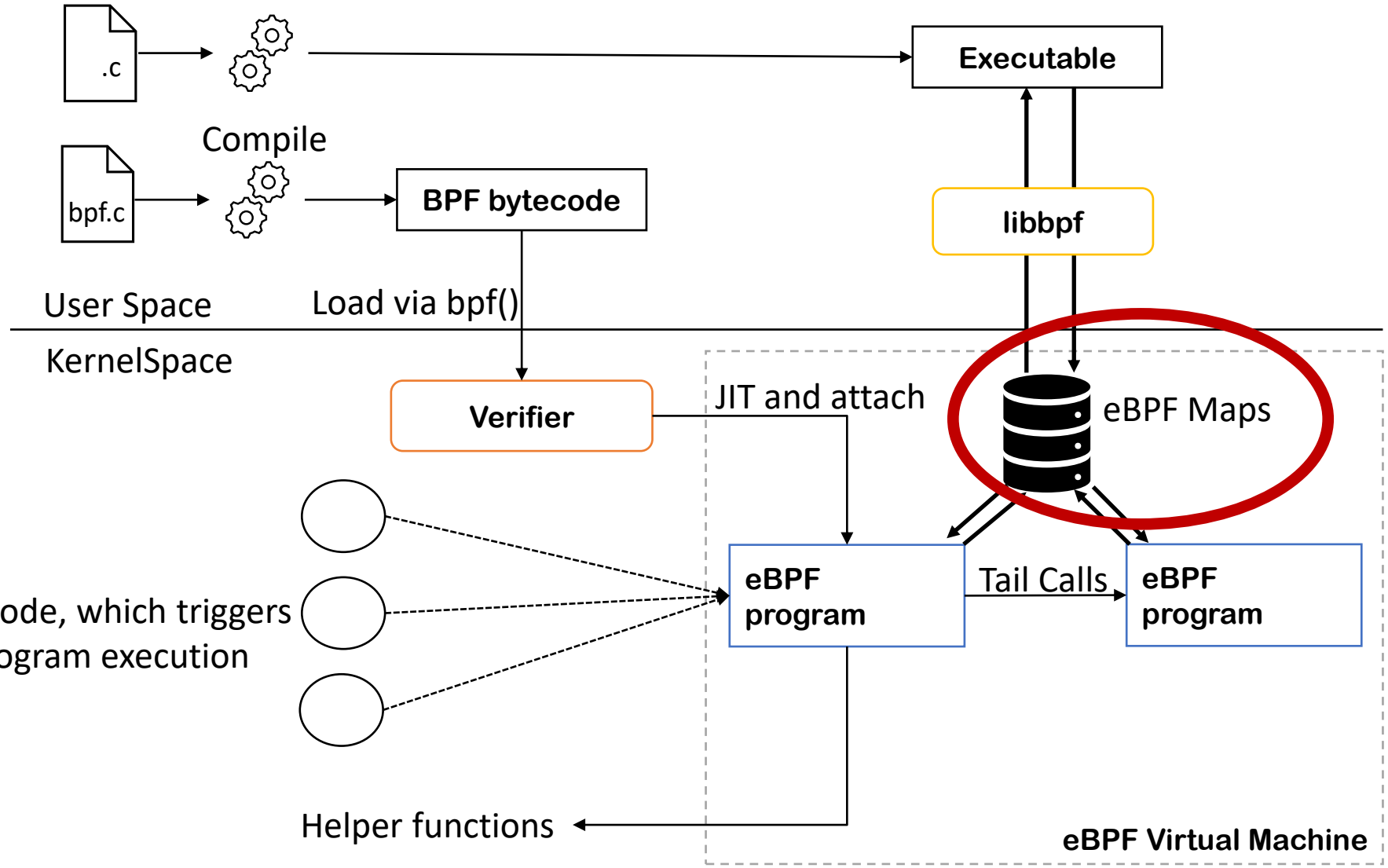
eBPF Program Types

- Program types define where and when an eBPF program can be called
  - **Trigger:** When/where the program runs
  - **Context:** What data the program can access
- **Kprobes/Kretprobes:**
  - Can be attached to almost any kernel function
  - Get the register state (pt_regs) as context i.e, the CPU register state
  - Alternative: Fentry/Fexit
- **Tracepoints:**
  - Special locations in the kernel
    - *cat /sys/kernel/debug/tracing/available_events*
  - Context depends on tracepoint
    - *cat /sys/kernel/debug/tracing/events/<subsystem>/<event>/format*
- **Uprobes/Uretprobe:**
  - Attach eBPF programs to user space programs/libraries

eBPF Program Types

- Linux Security Modules (LSM):
  - Allow security checks (see AppArmor or SELinux)
  - eBPF return value can override a decision
    - Allow or deny a request
- Traffic Control (TC):
  - Enables traffic shaping, traffic scheduling
  - Socket Buff as context
  - Return value decides the packet's fate
    - Drop or pass
- eXpress Data Path (XDP):
  - XDP hook directly after receiving a network packet
  - Raw frame and some metadata as context
  - Return value defines a decision for the packet
    - Drop, Accept, Redirect

eBPF Maps



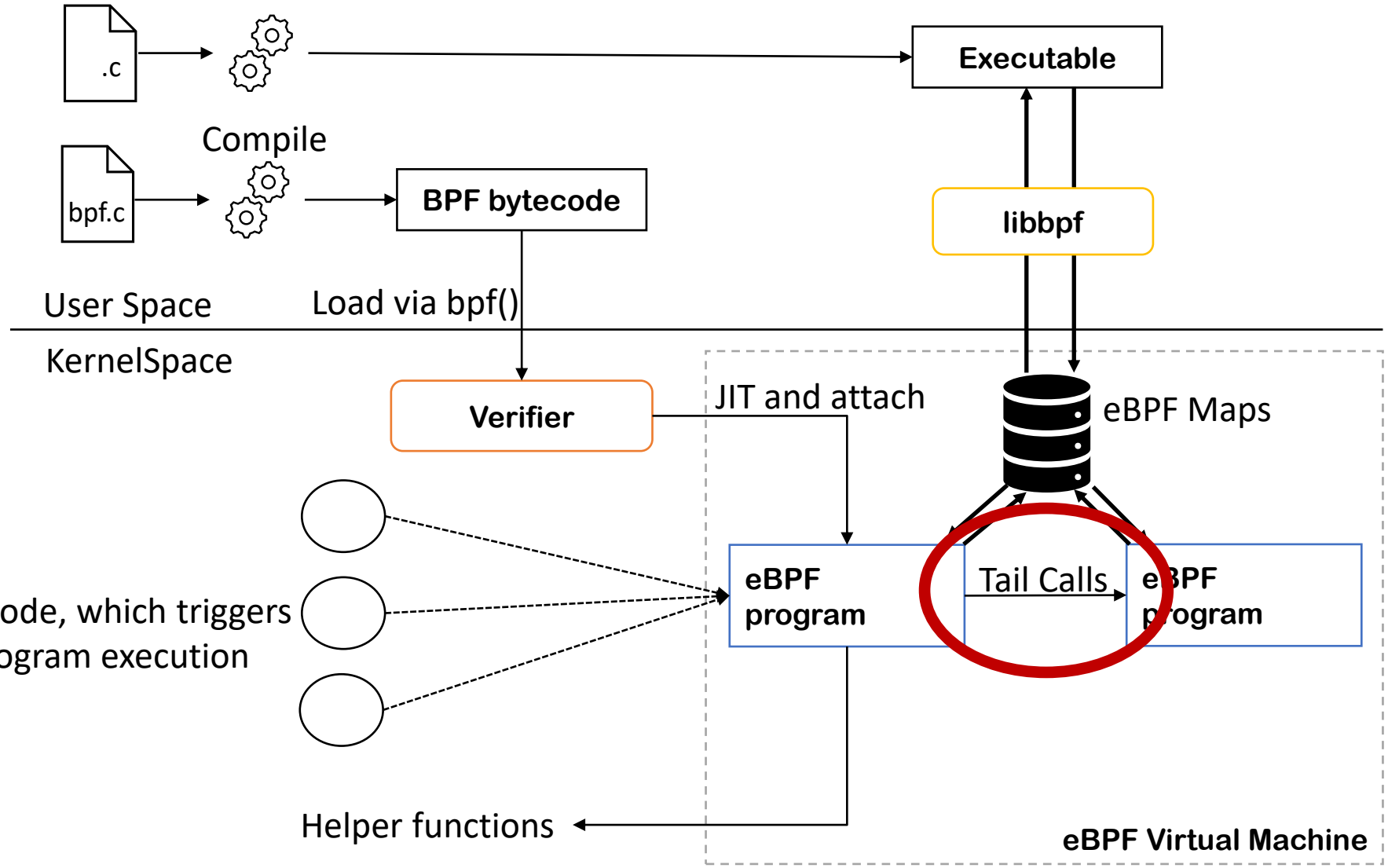
Hooks:
Kernel code, which triggers eBPF program execution

eBPF Maps

- Maps provide **Persistent state** for eBPF programs
- Kernel-Space - User-Space Communication
  - Kernel-resident data structures that can be accessed by eBPF and user-space programs
- Maps are declared in the eBPF program and automatically allocated and destroyed depending on the eBPF program's lifetime
- Generally, all maps are key-value stores but different types exist:
  - Arrays
  - Queues
  - Stacks
  - LRU
  - LPM
  - Bloom Filters, …

```
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, int);
    __uint(max_entries, 16);
} kv_store SEC(".maps");
```

eBPF Tail Calls

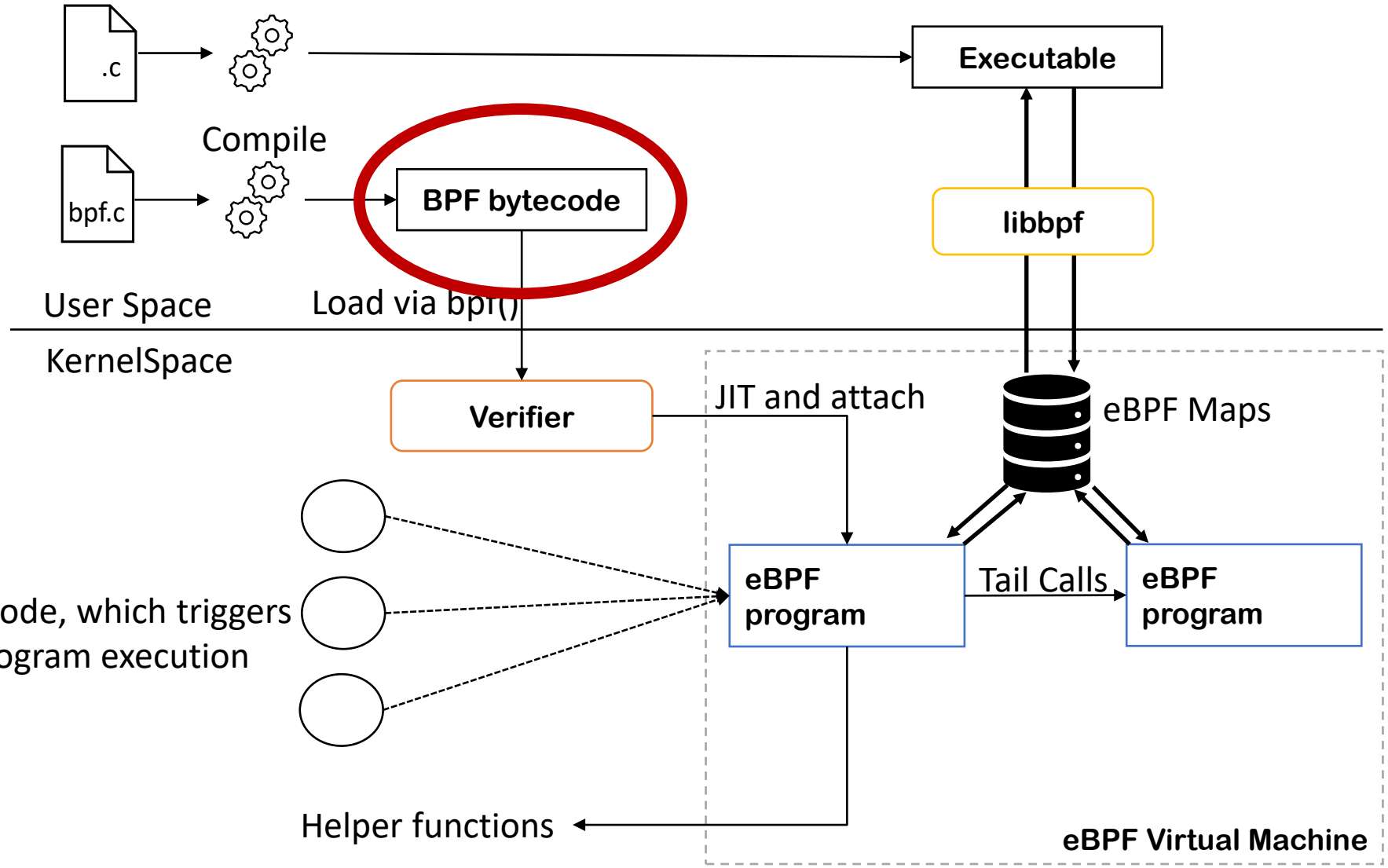


Hooks:
Kernel code, which triggers eBPF program execution

eBPF Tail Calls

- Tail calls allow one eBPF program to call another eBPF program and replace the current execution context
- Execution does not return to caller once tail call completes
  - **Jump to another eBPF program (no return)**
- Uses a map of type PROG_ARRAY to hold file descriptors of available eBPF programs
- e.g., Prog A → Prog B → Prog C
- Tail calls turn eBPF into a programmable pipeline

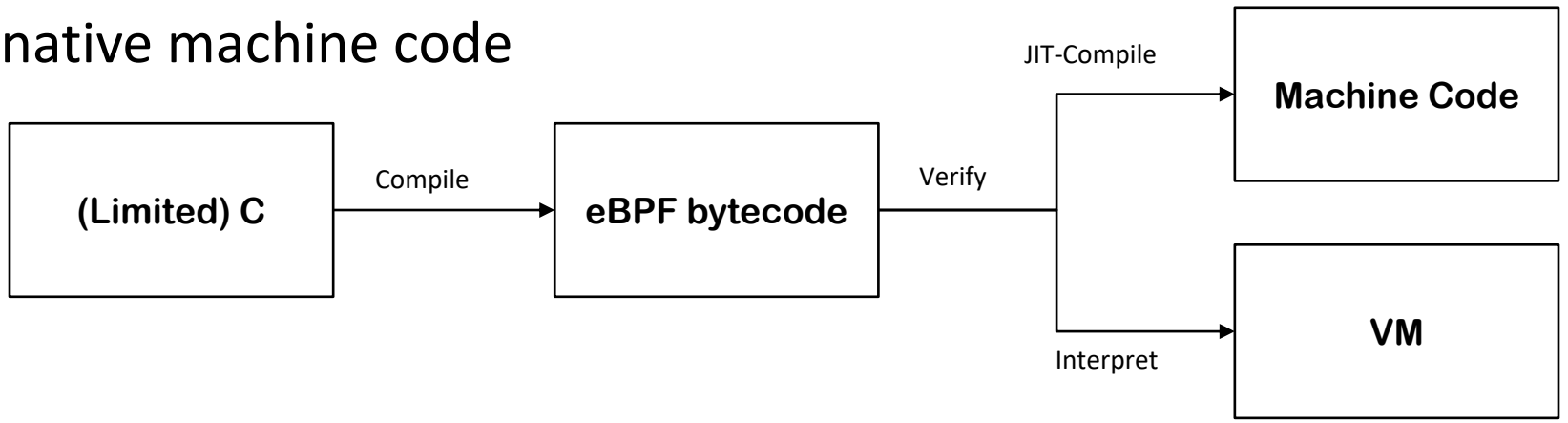
eBPF Architecutre



Hooks:
Kernel code, which triggers eBPF program execution

eBPF Bytecode

- eBPF specifies its own ISA
  - 10 general-purpose register
  - 64 bit instructions
- An eBPF program is a set of EBPF bytecode instructions
- Byte code is executed in the kernel either by
  - VM (interpreted)
  - JIT-compiled to native machine code



eBPF VM Model

```c
SEC("ksyscall/execve")
int BPF_KSYSCALL(my_probe, const char *filename)
{
    bpf_printk("Execve: %s\n", filename);
    return 0;
}
char LICENSE[] SEC("license") = "GPL";
```

llvm-objdump –S execve.bpf.o

```
execve.bpf.o:   file format elf64-bpf

Disassembly of section ksyscall/execve:

0000000000000000 <my_probe>:
; int BPF_KSYSCALL(my_probe, const char* filename) {
       0:       18 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 r2 = 0 ll
       2:       71 22 00 00 00 00 00 00 r2 = *(u8 *)(r2 + 0)
       3:       15 02 09 00 00 00 00 00 if r2 == 0 goto +9 <LBB0_2>
       4:       b7 02 00 00 70 00 00 00 r2 = 112
; int BPF_KSYSCALL(my_probe, const char* filename) {
       5:       79 13 70 00 00 00 00 00 r3 = *(u64 *)(r1 + 112)
       6:       0f 23 00 00 00 00 00 00 r3 += r2
       7:       bf a1 00 00 00 00 00 00 r1 = r10
; int BPF_KSYSCALL(my_probe, const char* filename) {
       8:       07 01 00 00 f8 ff ff ff r1 += -8
       9:       b7 02 00 00 08 00 00 00 r2 = 8
      10:       85 00 00 00 71 00 00 00 call 113
      11:       79 a3 f8 ff 00 00 00 00 r3 = *(u64 *)(r10 - 8)
      12:       05 00 01 00 00 00 00 00 goto +1 <LBB0_3>
```

Compile and Load

```
$ clang –target bpf \
        –I /usr/include/$(shell uname –m)-linux-gnu \
        -g –O2 –c execve.bpf.c –o execve.bpf.o
$ sudo bpftool prog load execve.bpf.o /sys/fs/bpf/execve
        # /sys/fs/bpf/execve is a pinned reference to an eBPF program in the kernel
```

- bpftool allows inspection and manipulation of eBPF objects
  - Load, list, inspect, pin, profile, attach
  - Create, dump, update, delete maps
  - Developer tools:
    - generate header file: vmlinux.h
      - Contains type definitions of the current kernel
    - Skeleton programs (see next slide)

* -O2 is important, some BPF programs won't work without it

Bpftool Generate Skeleton

```
$ bpftool gen skeleton execve.bpf.o > execve.skel.h
```

- Generates a header file to load, attach, detach and destroy the ebpf program
- Can be used in user-space program to manage the ebpf program
- Alternative to bpftool

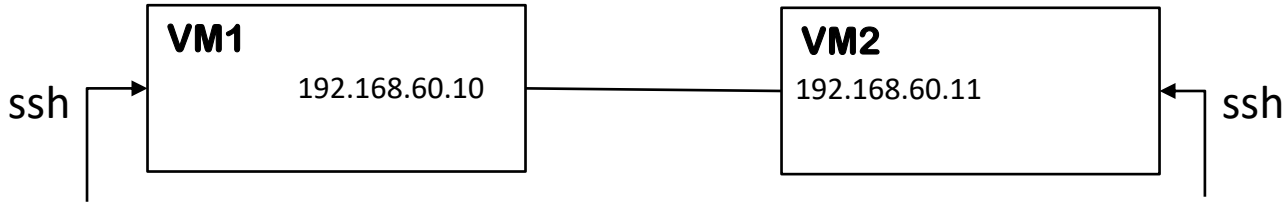
```c
#include "execve.skel.h"
int main(void)
{
    struct execve_bpf *prog = execve_bpf__open_and_load();
    execve_bpf__attach(prog);
    bpf_link__pin(prog->links.my_probe, "/sys/fs/bpf/execve");
    execve_bpf__destroy(prog);
}
```

Pinning eBPF objects

- eBPF objects (programs, maps) get destroyed when there is no reference to it
- To keep the eBPF program loaded and attached it can be pinned to the filesystem
- Once the program is removed, the reference counter (ref-counter) of the eBPF object is zero and is cleaned up by the kernel
- Pinning:
  - `bpf_link__pin(bpf_prog_link, "/sys/fs/bpf/execve")`
  - `bpftool prog load execve.bpf.o /sys/fs/bpf/execve`
- `Unload/Unpin:`
  - `$ sudo rm -rf /sys/fs/bpf/execve`

Test environment

- Go to directory with `Vagrantfile`
- Call `$ vagrant up`
- This creates 2 virtual machines and installs all dependencies



- Vagrant will automatically create a shared folder to your host system's folder where you called "`$ vagrant up`"
- Connect to the virtual machine via `$ vagrant ssh vm1`
- Vagrant will place the shared folder in "/vagrant"
- Take a look at the examples and Makefiles

Demo

Further Reading / Tools

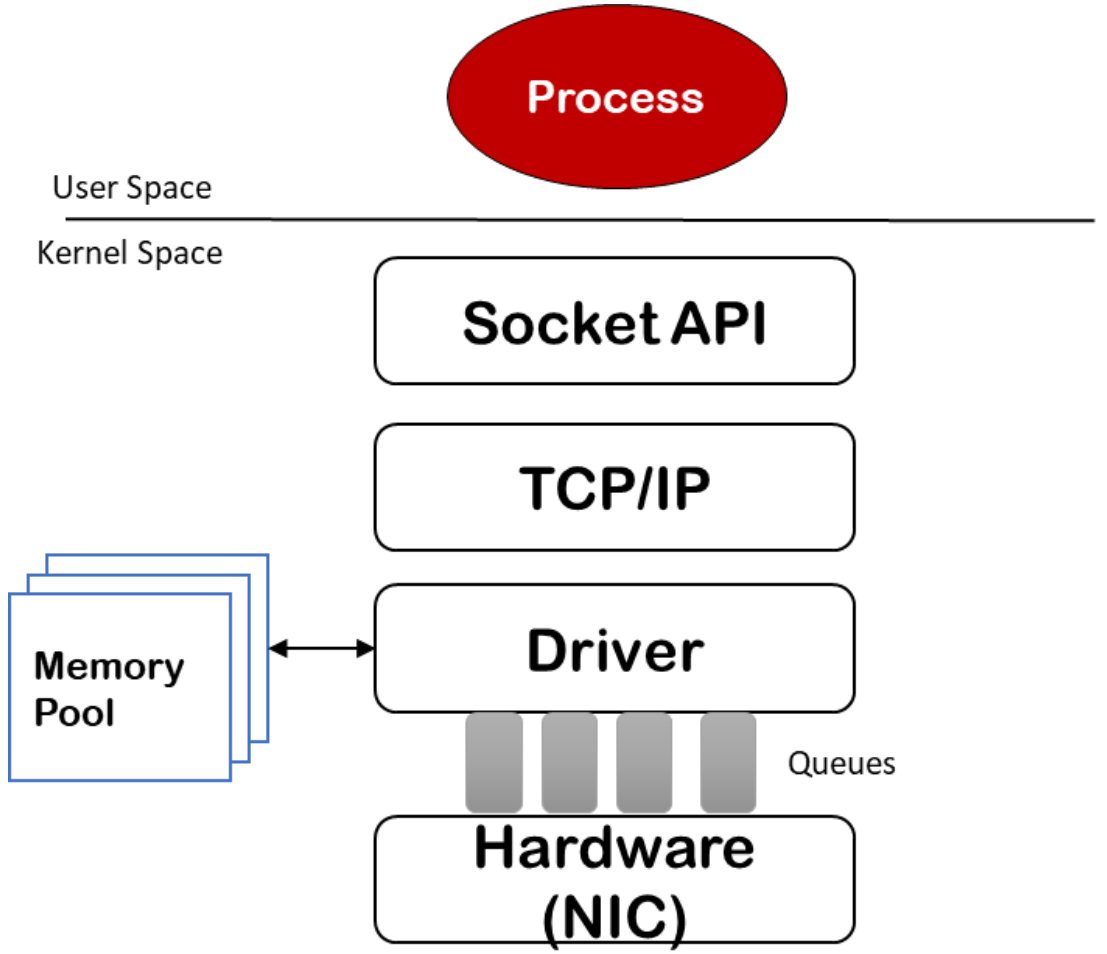
- Learning eBPF by Liz Rice
- XDP Tutorial
- bpftrace
- bcc
- BPF Documentary (youtube link)

RDMA

Remote Direct Memory Access

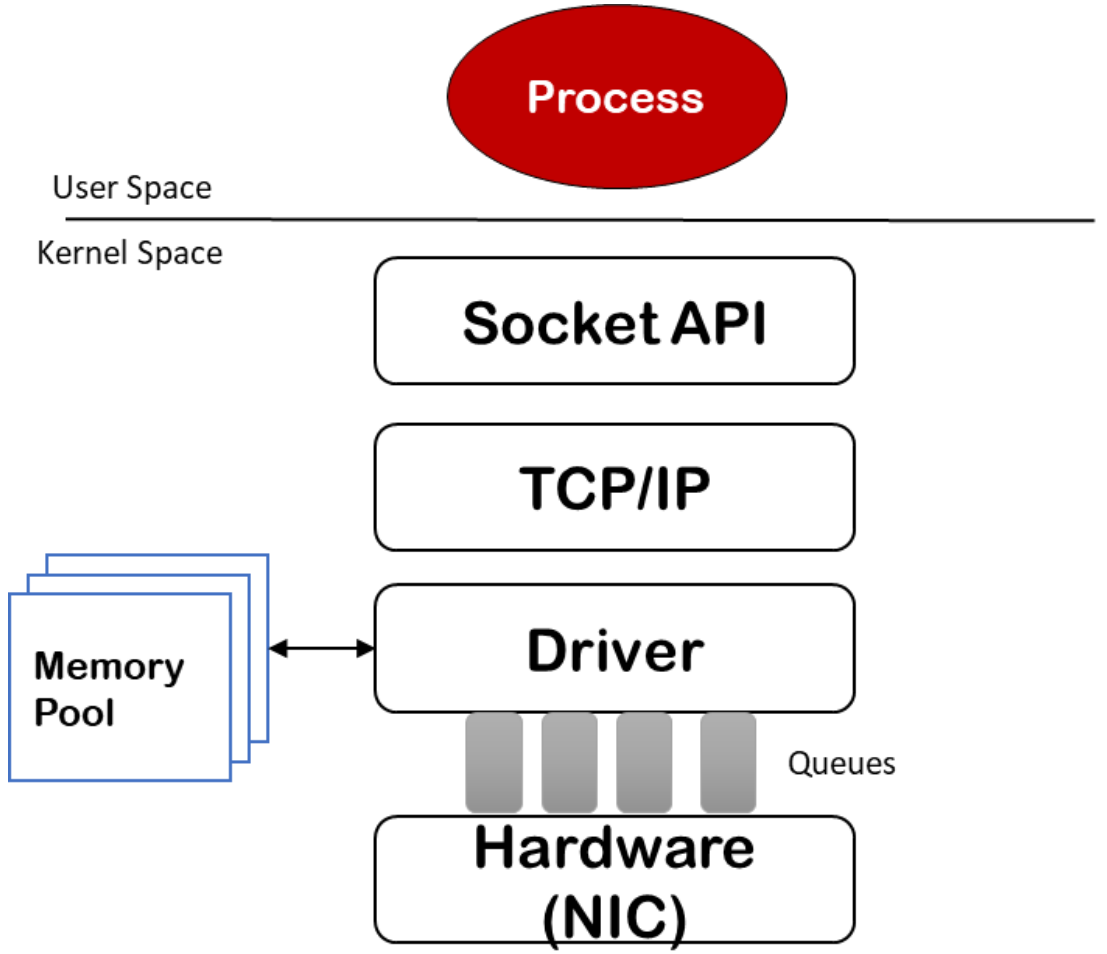
Reminder Kernel Networking

- Limited throughput
- Adds Latency
- High CPU utilization
- Overheads
  - System Calls
  - Buffering
  - Packet Parsing
  - Packet Classification
  - Interrupts



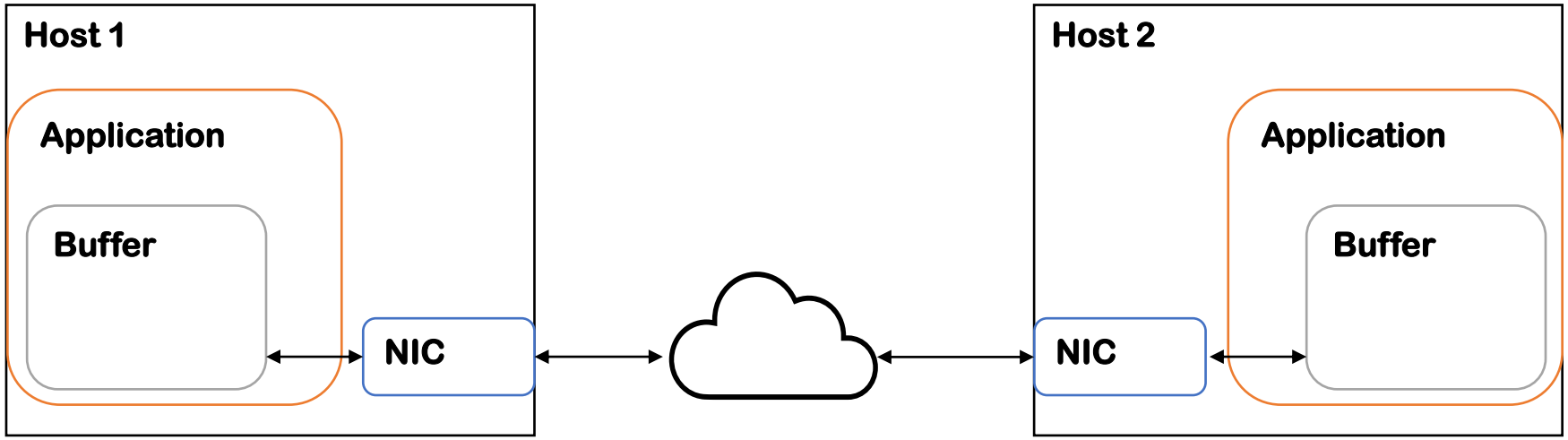
Reminder Kernel Networking

1. Application's data → user-space pages in DRAM
2. **Copy** to Kernel-space DRAM (Kernel socket buffers)
3. Kernel processing: TCP, IP…
4. Driver TX Ring
5. DMA read by NIC
6. NIC → Wire



RDMA

- Mechanism to directly access memory of a remote application
- Offload data transfer to specialized hardware
  - Reduces CPU load
  - Bypasses kernel
  - Asynchronous operations
- Achieves high throughput and low latencies

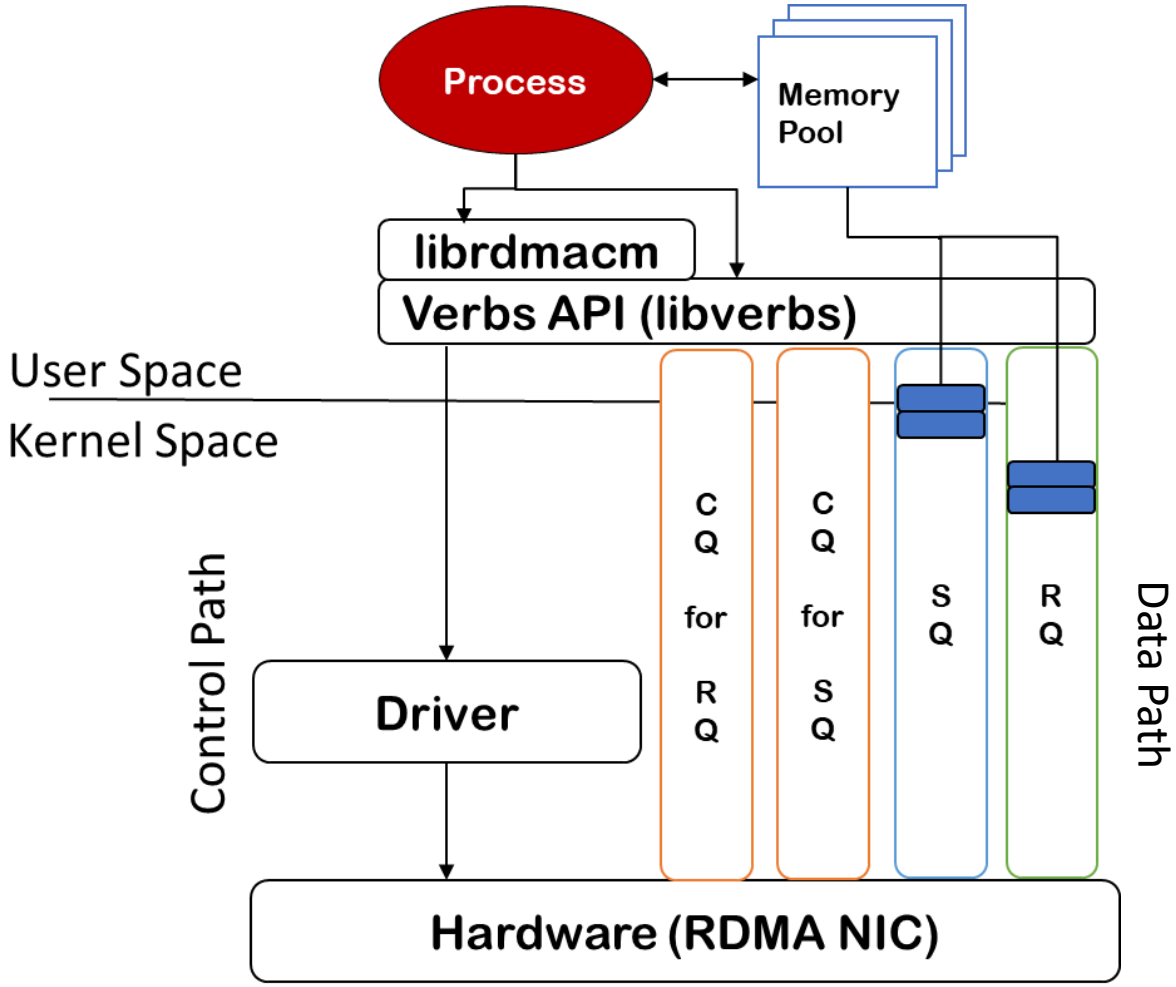


Implementations of RDMA

- Physical / Link Layer
  - Infiniband (IB)
    - Proprietary protocol stack (Nvidia)
  - RoCE
    - RDMA over Converged Ethernet
    - Uses Ethernet link layer
    - RoCEv2: RDMA over IP/UDP
- Transport modes:
  - Reliable Connection (RC)
  - Unreliable Connection (UC)
  - Unreliable Datagram (UD)

RDMA Network Stack

- Control path configures hardware
- Data path is organized through queues
- QP: Queue Pair
- RQ: Receive Work Queue
- SQ: Send Work Queue
- QP = RQ + SQ
- CQ: Completion Queue



RDMA queues

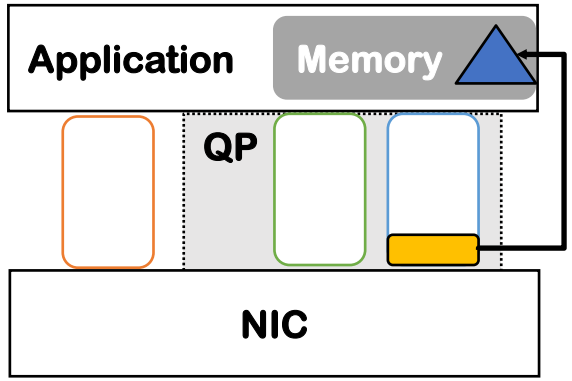
- Queue Pairs are the interface between application and NIC
- Queue Pairs define the type of connection
  - Reliable Connection (RC)
  - Unreliable Connection (UC)
  - Unreliable Datagram (UD)
- Queue Pairs are used to post work requests (WR)
- WR define the operation that should be performed by the NIC
- Multiple WR can be queued up
- Once the NIC completed a WR it places a Completion Queue Element (WQE) into the respective completion queue

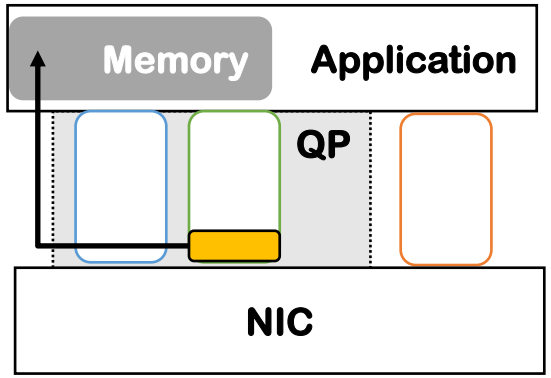
RDMA Operations (Verbs)

- Channel Operations
  - SEND
    - Send data to remote node
  - RECV
    - Receive data from remote node
- Memory Operations
  - WRITE
    - Write data to specified address at remote machine
  - READ
    - Read data from specified address at remote machine
  - ATOMIC
    - Atomic fetch-add and compare-swap on specified address at remote machine

RDMA SEND/RECV

- Sender posts work request (WR) in its send queue (SQ)
- Receiver posts work request in ist receive queue (RQ)

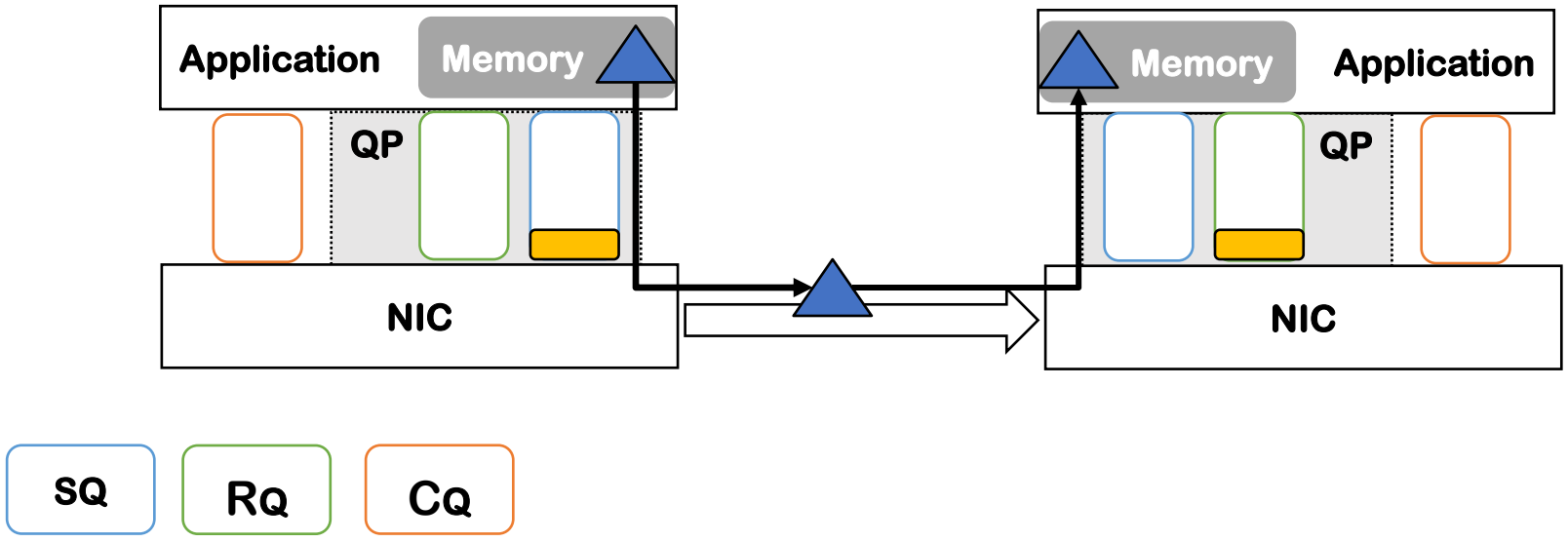






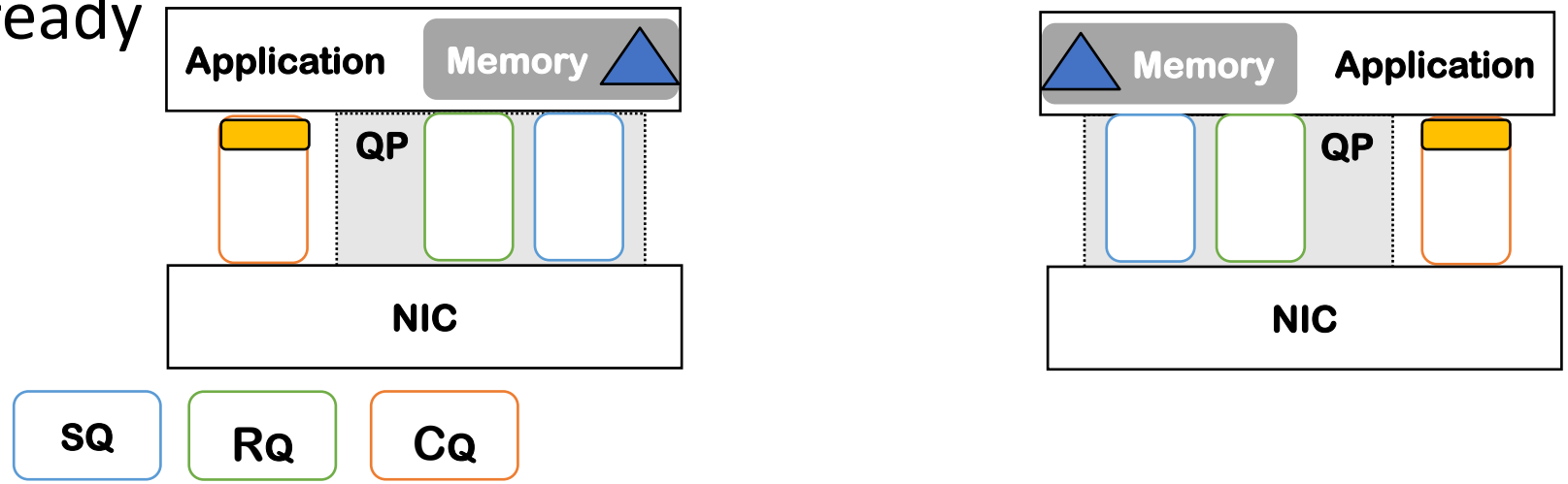
RDMA SEND/RECV

- Sender posts work request (WR) in its send queue (SQ)
- Receiver posts work request in ist receive queue (RQ)
- NIC transmits the data



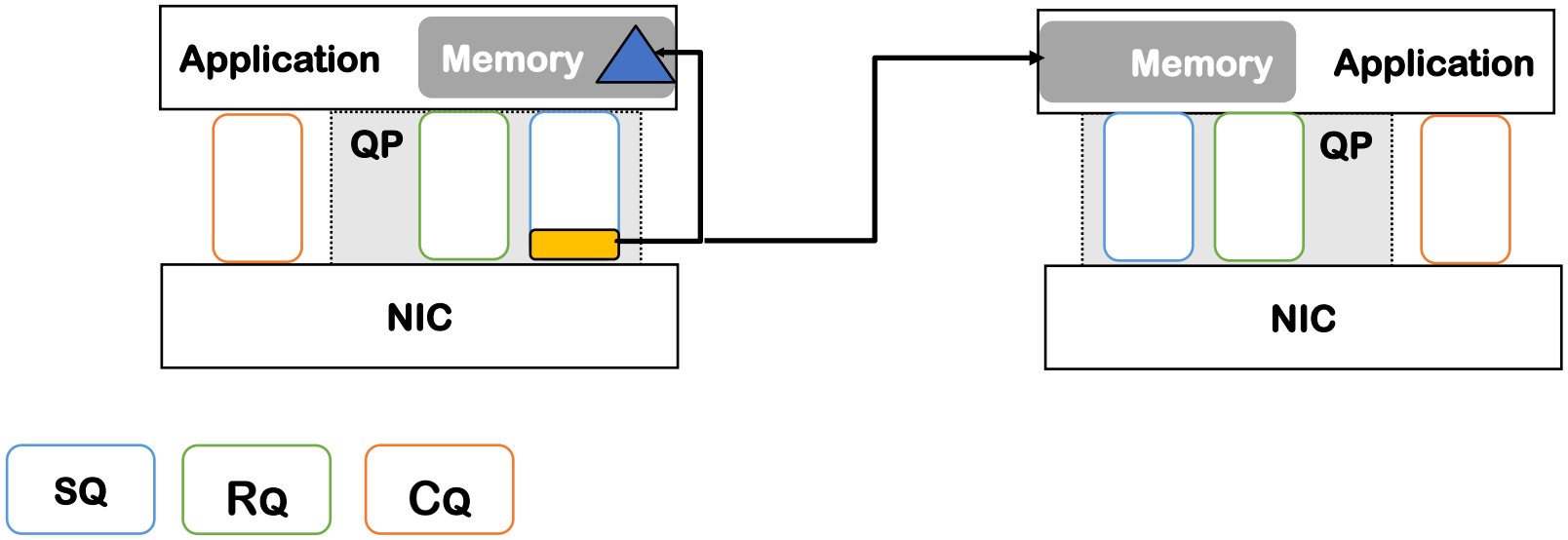
RDMA SEND/RECV

- Sender posts work request (WR) in its send queue (SQ)
- Receiver posts work request in ist receive queue (RQ)
- Sender‘s NIC places CQE into CQ-SQ to inform the sender that the data was transmitted
- Receiver‘s NIC places CQE into CQ-RQ to inform the receiver that the data is ready



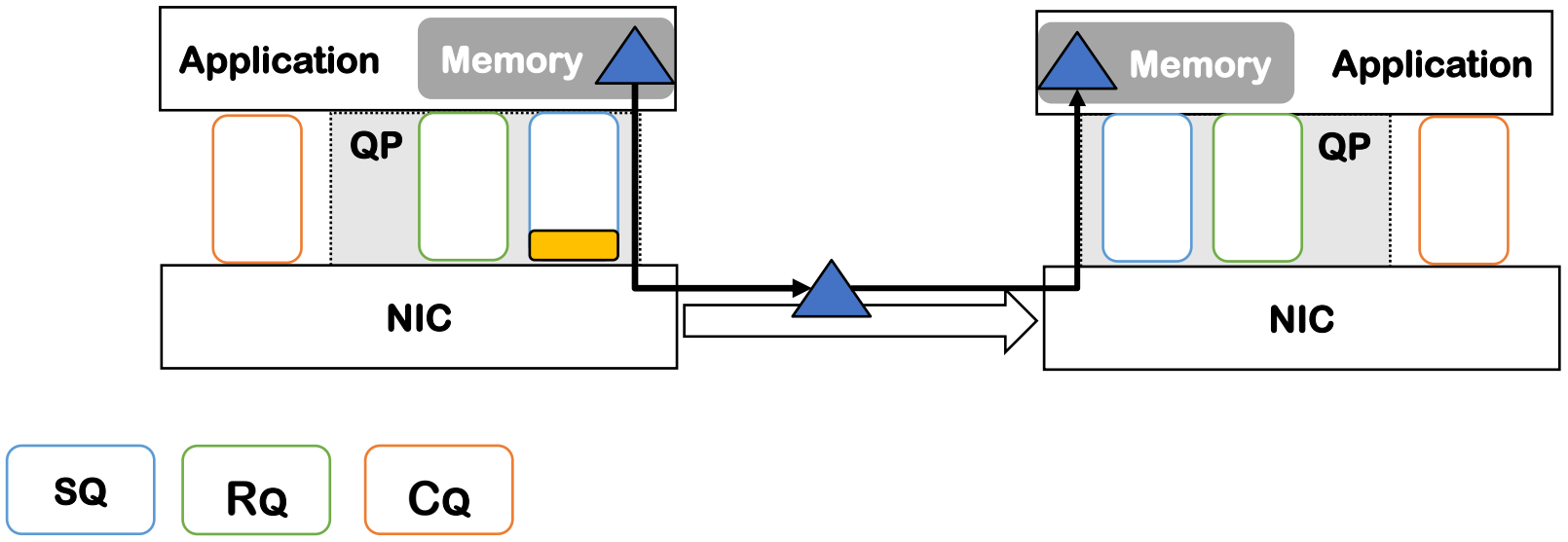
RDMA WRITE

- Sender posts work request (WR) in its send-queue (SQ)
- Work request needs virtual address and identifier (key) of remote buffer



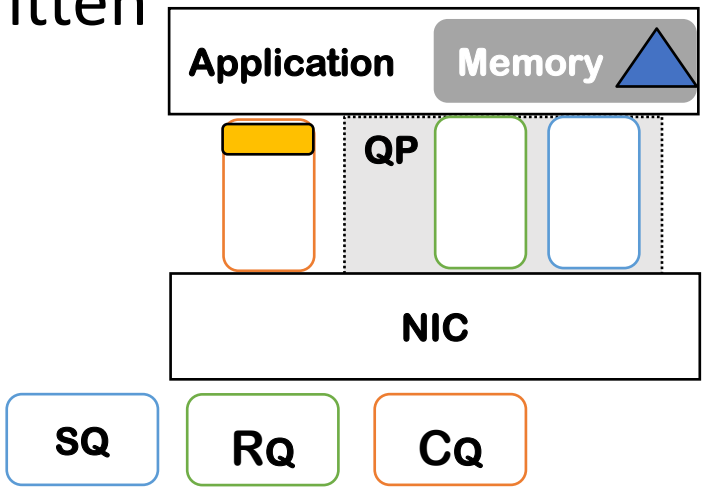
RDMA WRITE

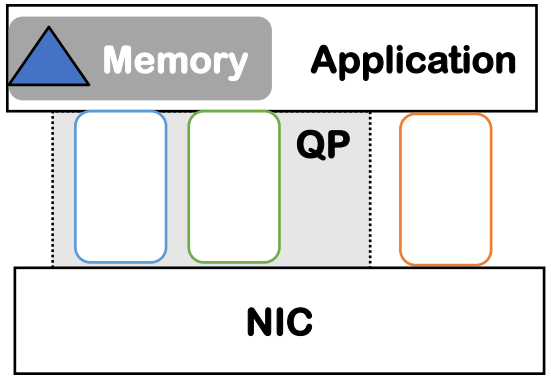
- Sender posts work request (WR) in its send queue (SQ)
- Work request needs virtual address and identifier (key) of remote buffer
- NIC issues write to remote memory



RDMA WRITE

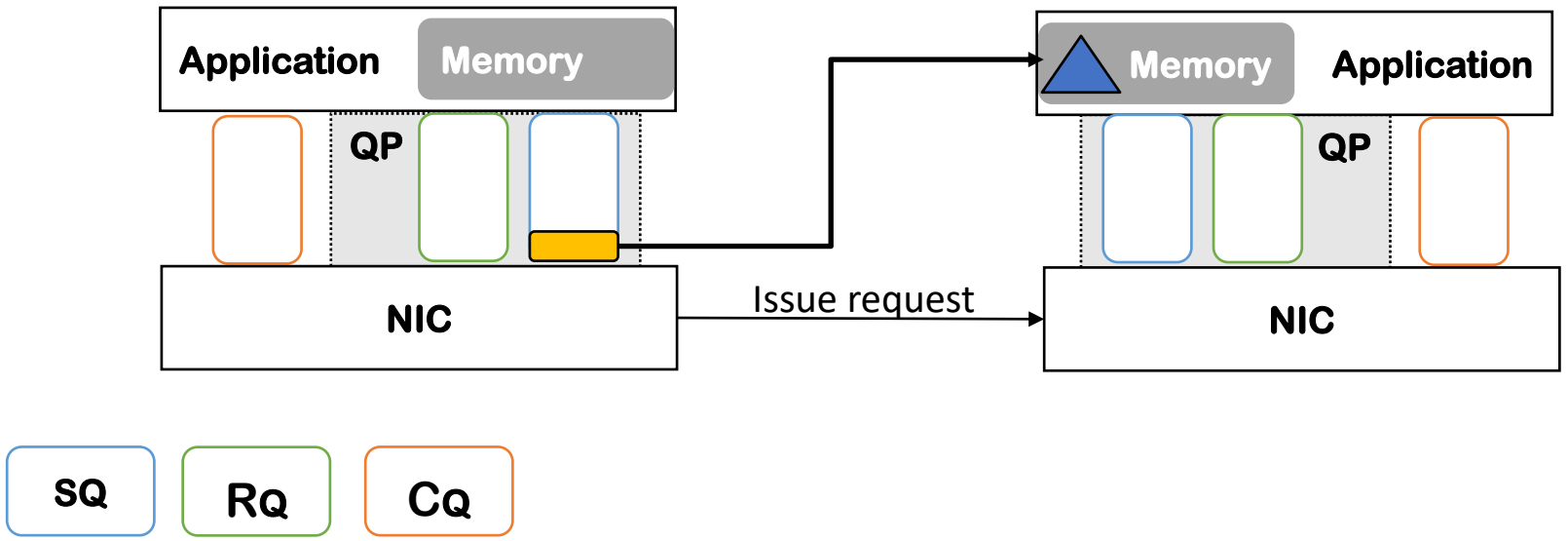
- Sender posts work request (WR) in its send queue (SQ)
- Work request needs virtual address and identifier (key) of remote buffer
- NIC issues write to remote memory
- Sender‘s NIC places CQE into CQ-SQ to inform the sender that the data was written





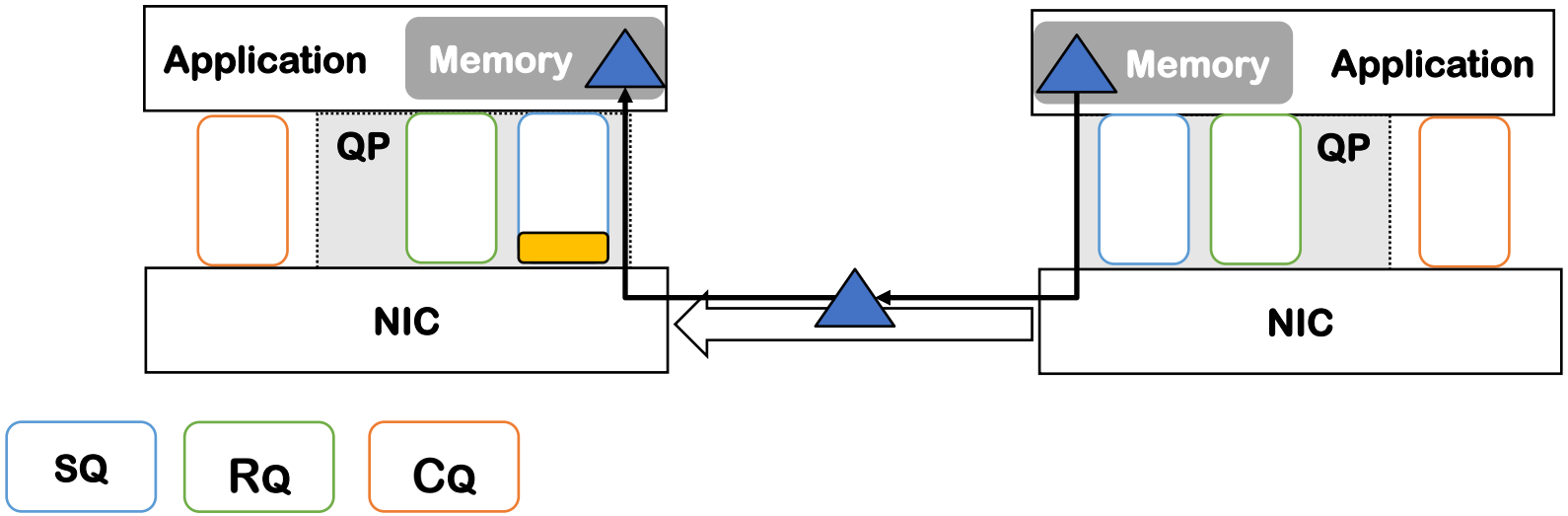
RDMA READ

- Sender posts work request (WR) in its send queue (SQ)
- Work request needs virtual address and identifier (key) of remote buffer
- Sender sends request to remote NIC



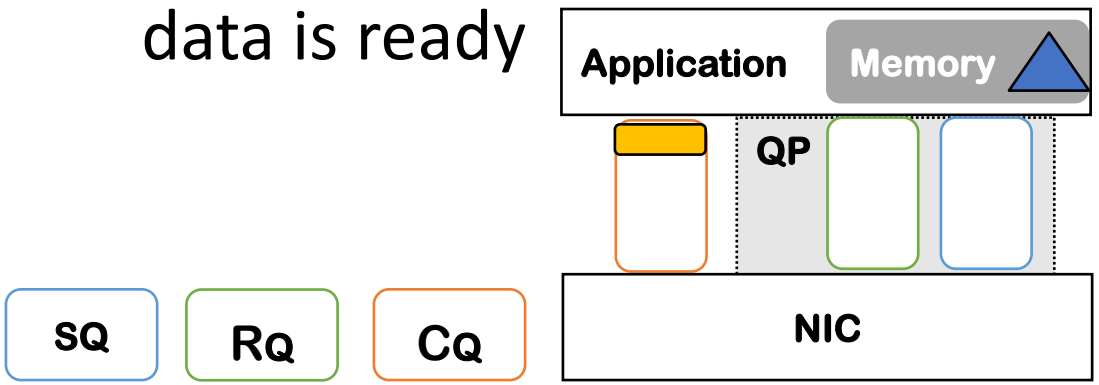
RDMA READ

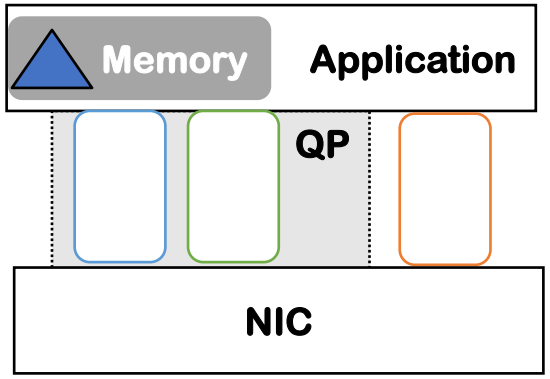
- Sender posts work request (WR) in its send queue (SQ)
- Work request needs virtual address and identifier (key) of remote buffer
- Sender sends request to remote NIC
- Remote NIC fetches the data and sends it to the Sender



RDMA READ

- Sender posts work request (WR) in its send queue (SQ)
- Work request needs virtual address and identifier (key) of remote buffer
- Sender sends request to remote NIC
- Remote NIC fetches the data and sends it to the Sender
- Sender‘s NIC places CQE into CQ-SQ to inform the sender that the data is ready





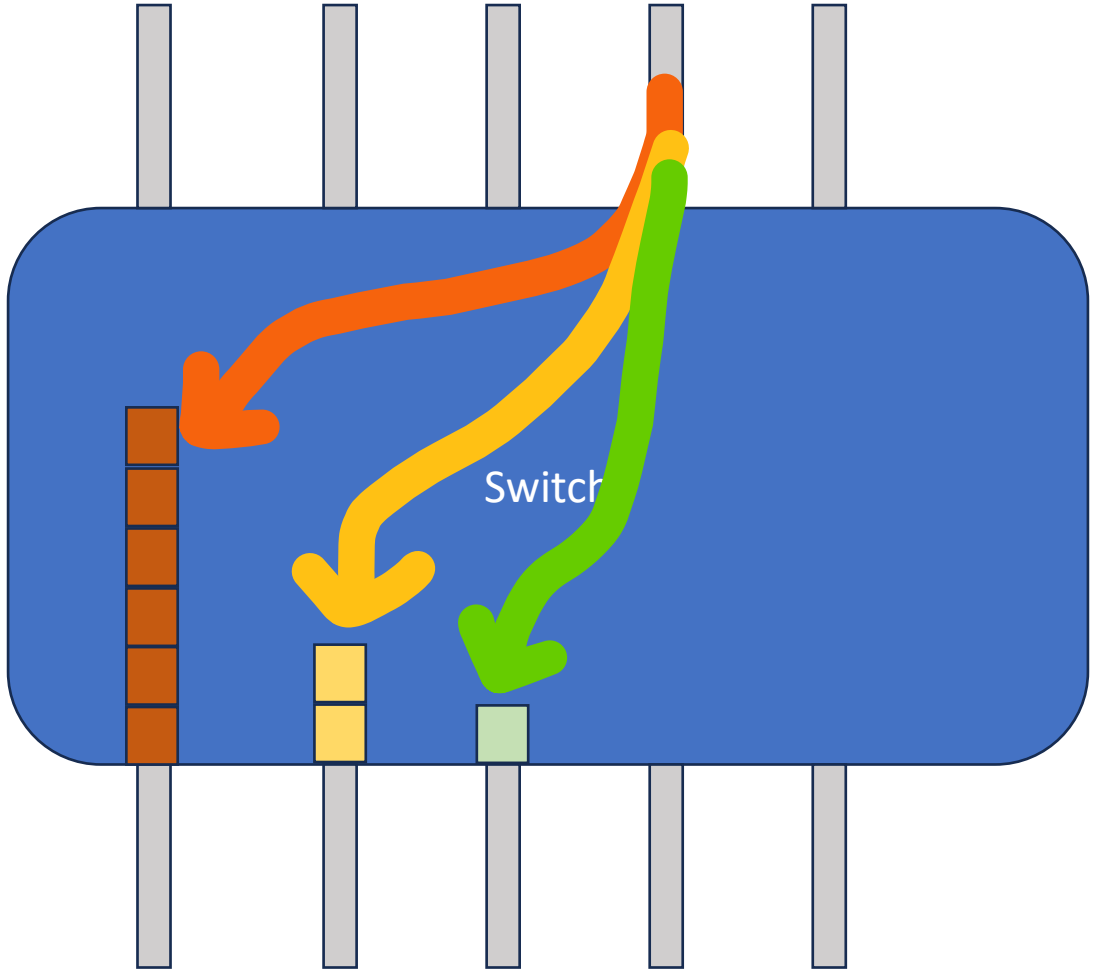
RDMA Congestion Control

- Implemented on RDMA NIC
- Simple Go-back-N algorithm when packet loss is detected
- NACK upon out-of-order packets
- Bad performance if too many packets are lost
- RDMA performs best if the network/fabric is lossless
- Infiniband uses a credit-based flow control to avoid packet drops
- PFC (Priority Flow Control) for RoCE

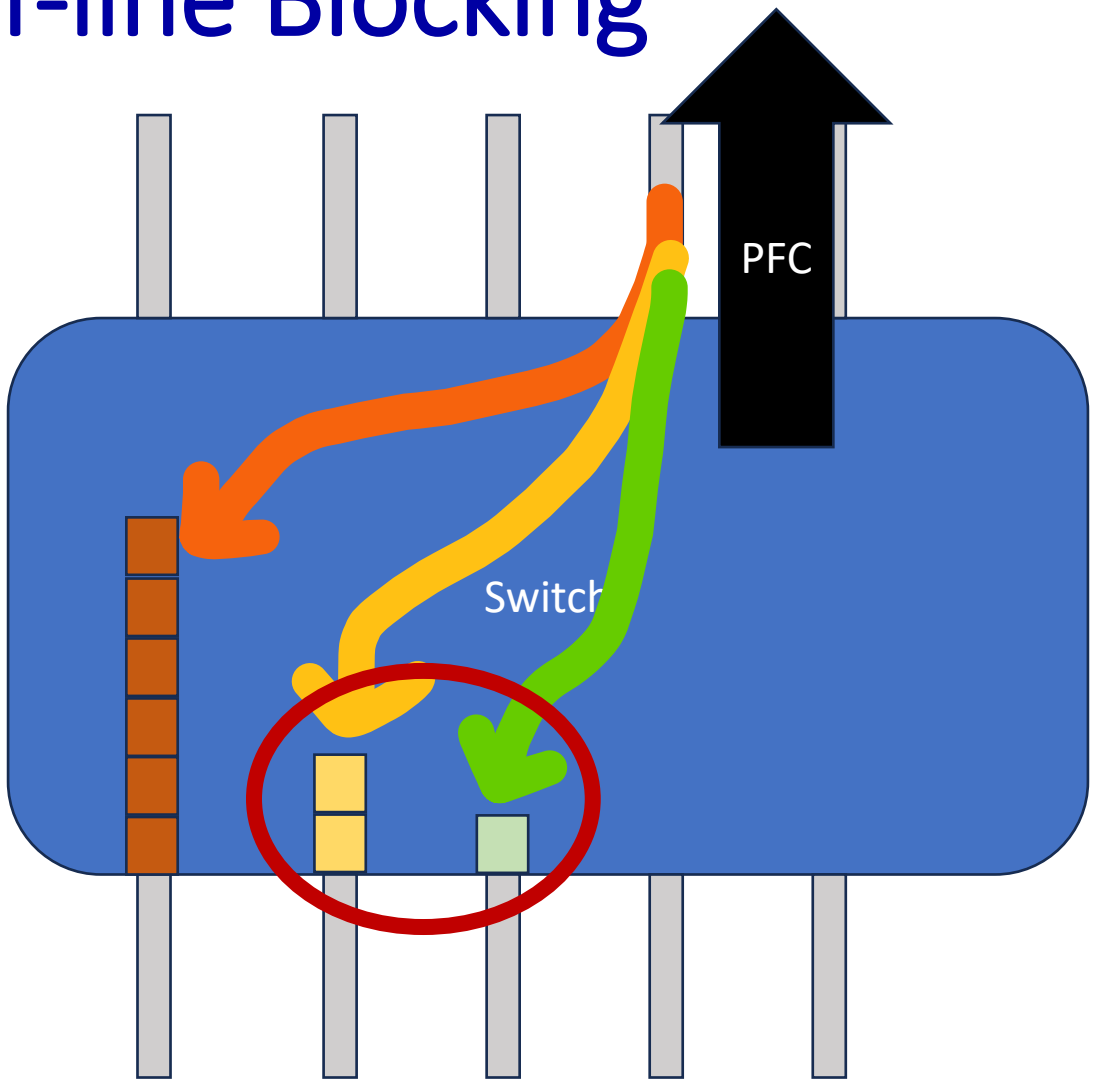
Go-Back-0 & Go-Back-N

- Upon a packet drop or out-of-order packet
  - Receiver sends a NACK
  - Sender starts retransmitting from 0 again
- Result → A flow only completes if it never experiences a packet drop in a continuous sequence of transmissions from 0 to the end
  - A flow may get struck in forever retransmissions: **Live lock!**
- Go-Back-**N** is simple and solves the problem
  - Retransmit only from the last lost packet
  - Different from TCP: Go-Back-N simply *restarts* transmitting all data from starting from the last lost packet

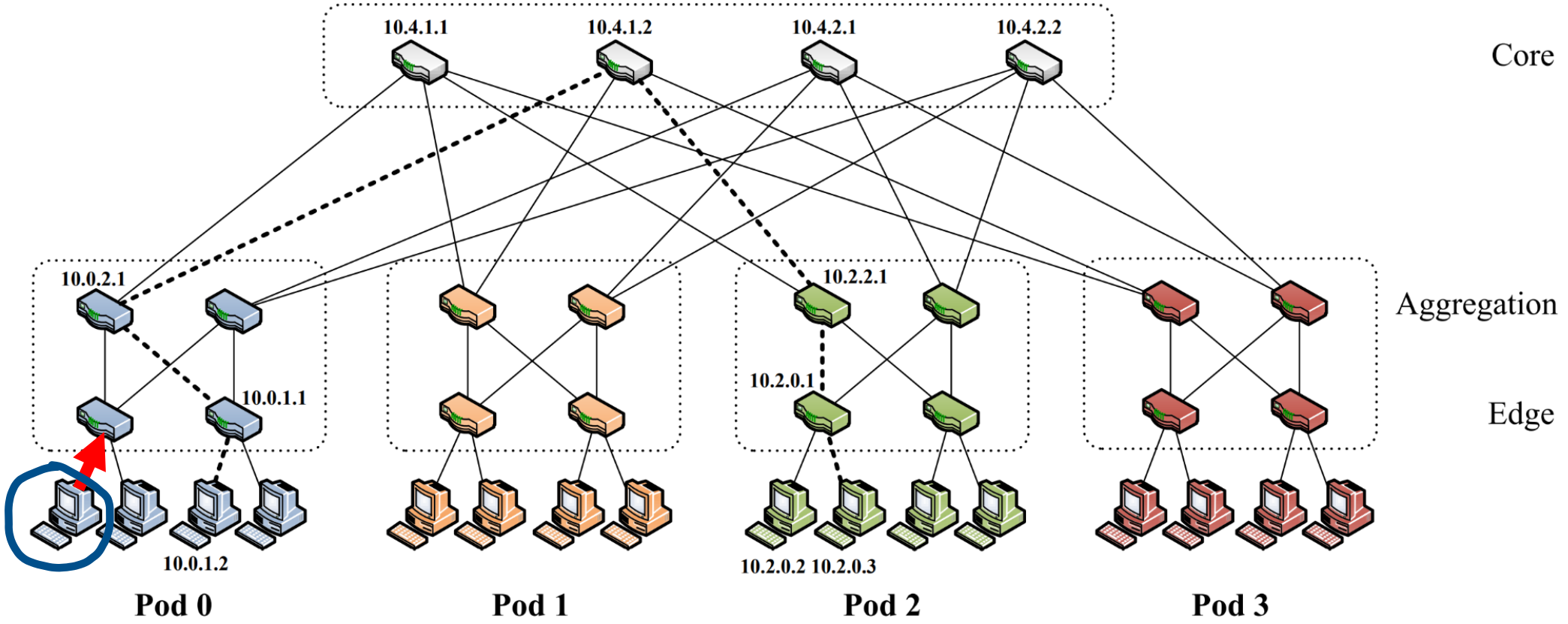
PFC Head-of-line Blocking



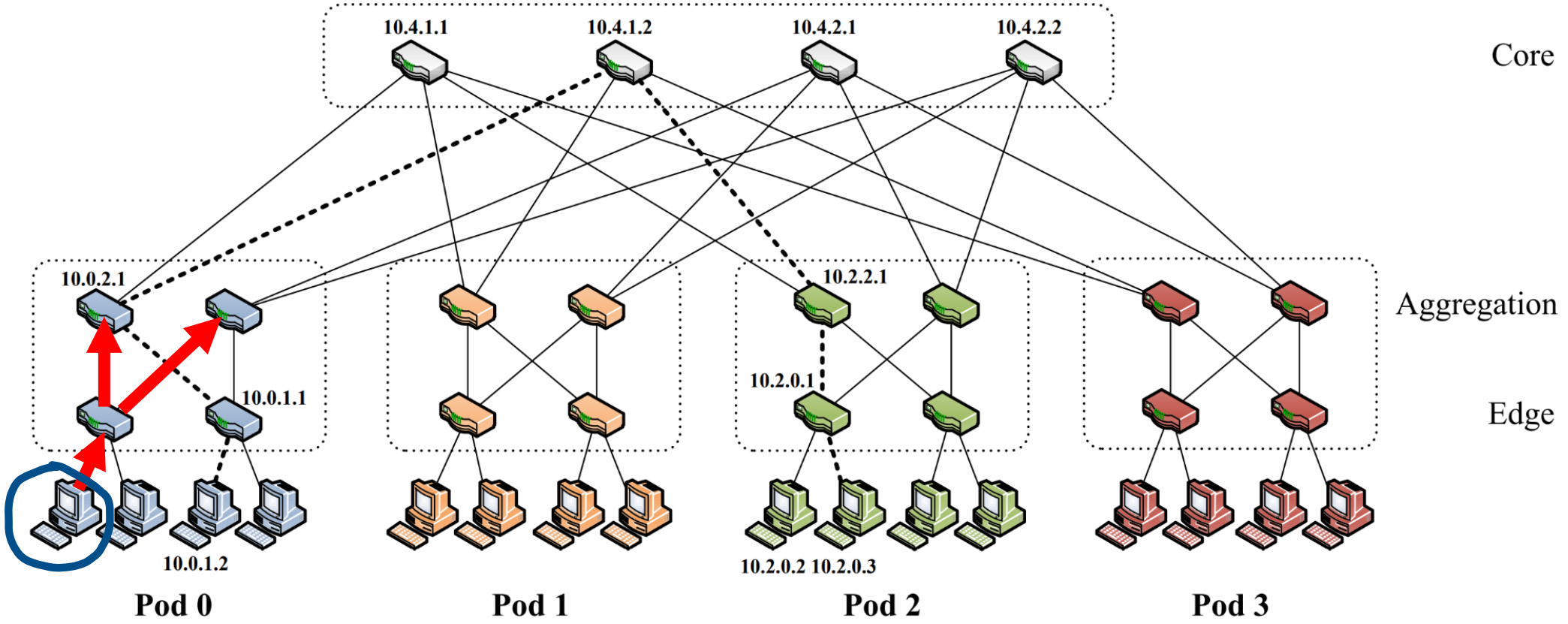
PFC Head-of-line Blocking



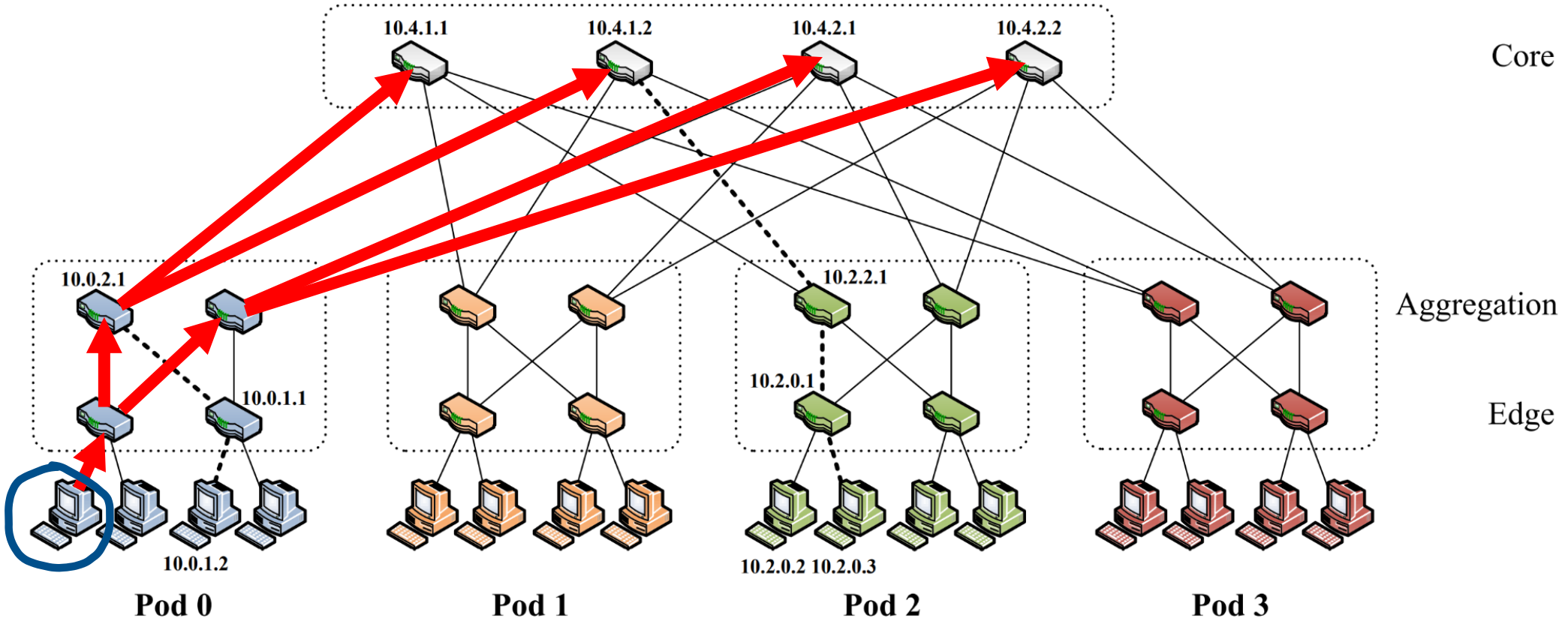
PFC Congestion Spreading



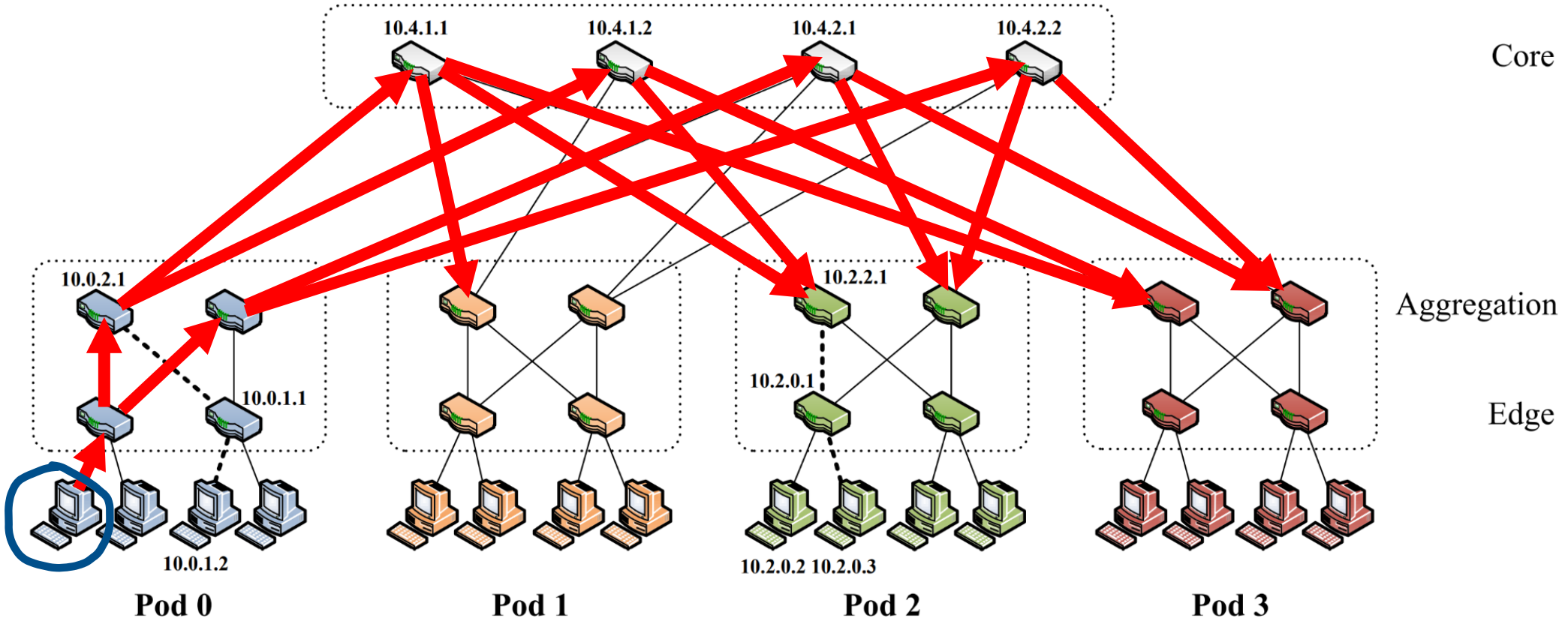
PFC Congestion Spreading



PFC Congestion Spreading



PFC Congestion Spreading

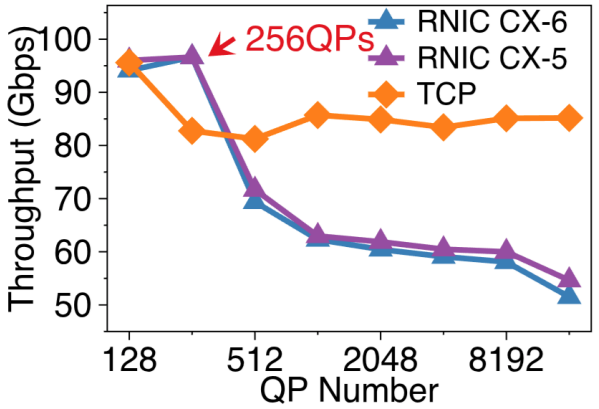


PFC Deadlocks

- Congestion spreading can potentially lead to deadlocks
- Root cause: A cyclic dependency across buffers

RDMA Queue Pairs

- Notice that each “flow” has its queue pair (SQ and RQ)
- Queue pairs consume resource on hardware
- Excessive number of QPs can result in paging → lower throughput
- Think about All to All communication in a large training cluster
  - QP state explosion!
  - How to efficiently limit the number of active QPs while ensuring fast all to all completion?



SRNIC: A Scalable Architecture for RDMA NICs

Further Reading

- Revisiting Network Support for RDMA
  - Good background section on Congestion Control for RoCE
- RDMA over Commodity Ethernet at Scale
- Congestion Control for Large-Scale RDMA Deployments
- Prevention of Deadlocks and Livelocks in Lossless,Backpressured Packet Networks
- RDMA over Ethernet for Distributed Training at Meta Scale

End of lecture

ib_send_bw